10–12 Minute Presentation Guide & Speaker Notes — FinDoc-AuditEngine v3.0

Companion Files in `docs/`:

- **PowerPoint Presentation (`.pptx`):** `FinDoc_AuditEngine_Presentation_Slides.pptx` (12 Widescreen 16:9 Slides with Speaker Notes)
- **Landscape Slide Deck (`.pdf`):** `FinDoc_AuditEngine_Slide_Deck.pdf` (12 Designed 16:9 Slides)
- **Recorded Demo Walkthrough Video (`.mp4`):** `FinDoc_AuditEngine_Demo_Walkthrough.mp4` (HD 720p Walkthrough of Dashboard, 3 Scenarios, Terminal Logs & Tests)

Recommended 10–12 Minute Presentation Timeline

Time Segment	Duration	Presentation Section	Slides Covered	Key Demonstration / Narrative Focus
Part 1	**1 Min** (`00:00–01:00`)	**Problem & Solution**	Slides 1–2	Why 3-Way Invoice–PO–GRN reconciliation, regex PII masking, and anomaly scoring are useful together.
Part 2	**2 Min** (`01:00–03:00`)	**System Architecture**	Slides 3–5	FastAPI pipeline: regex privacy masking, deterministic 3-way matcher, seeded synthetic Isolation Forest (`N=1,000`), approval router, and in-memory prototype ledger.
Part 3	**5 Min** (`03:00–08:00`)	**Live Demonstration (3 Scenarios)**	Slides 6–8	Live walkthrough of **Scenario 1 (`INV-1001` Clean Match)**, **Scenario 2 (`INV-1004` Price Discrepancy)**, and **Scenario 3 (`INV-1005` Critical-Risk Anomaly)** + terminal output.
Part 4	**2 Min** (`08:00–10:00`)	**Testing & Challenges**	Slides 9–10	Show `python -m pytest tests -q` (`28 passed` in Day 29 / `33 passed` in Day 30 — all collected tests passed) and explain the DOM decoupling & scaling fixes.
Part 5	**1–2 Min** (`10:00–12:00`)	**Limitations & Future Work**	Slides 11–12	Discuss in-memory ledger limitation, synthetic ML baseline calibration needs, security/RBAC controls, and PostgreSQL/OCR roadmap.

Detailed Speaker Script by Section

1. Problem and Solution (1 Minute — Slides 1–2)

- *"Hello everyone. Today I am presenting* `FinDoc-AuditEngine v3.0`*, our internship capstone prototype for Accounts Payable reconciliation and anomaly detection."*
- *"In procurement, finance teams must verify that every Vendor Invoice matches both the contractual Purchase Order (PO) and the warehouse Goods Receipt Note (GRN). Pure rule-based checks catch explicit unit-price or quantity mismatches, but they cannot flag an invoice that matches a PO on paper while exhibiting unusual risk signals—such as an offshore vendor demanding a 1-day rush payment just below a $50,000 sign-off limit."*
- *"Our prototype addresses this by combining pre-inference regex PII masking, deterministic 3-way SKU reconciliation, and a 7-feature Isolation Forest anomaly scorer with local perturbation explanations."*

2. Architecture (2 Minutes — Slides 3–5)

- *"Every request to* `POST /api/v1/audit/process` *passes through four modular stages:"*

1. *"First,* `FinancialPrivacyScrubber` *applies 6 regex patterns to mask sensitive identifiers like Indian PAN cards, Aadhaar-formatted numbers, bank accounts, phones, and emails before logging."*
2. *"Second,* `ReconciliationMatcher` *performs deterministic 3-way matching across SKU unit prices and net accepted dock quantities (*`received minus rejected`*) within a* `$0.05` *subtotal tolerance."*
3. *"Third,* `AnomalyScorer` *transforms 7 engineered features through* `StandardScaler` *and a 200-tree* `IsolationForest` *trained on a reproducible seeded synthetic baseline of 1,000 records. It outputs a* `0–100` *engineered anomaly-risk indicator and calculates single-feature perturbation attributions."*
4. *"Fourth,* `ApprovalRouter` *assigns the invoice to one of five governance queues and records the decision in our prototype's in-memory* `AUDIT_LEDGER` *list, displaying totals in both USD and INR (*`1 USD = 86.50 INR`*)."*

3. Live Demonstration — Three Representative Scenarios (5 Minutes — Slides 6–8)

- **Scenario 1 — Clean Invoice (`INV-1001`, `OfficeDepot India Pvt Ltd`, `$432.00` / `INR 37,368.00`):**
  - *"Let's load Scenario 1 in the dashboard. Two ergonomic chairs are billed at* `$180.00` *each, matching* `PO-9001` *exactly with zero dock rejections. Price variance is* `$0.00`*, the engineered risk indicator is low (*`25.0/100`*), and the router assigns* `TIER_1_STP_AUTO_APPROVED`*."*
- **Scenario 2 — Price Discrepancy (`INV-1004`, `Overcharge Technologies`, `$5,400.00` / `INR 467,100.00`):**
  - *"Next, let's test Scenario 2. Here, 10 enterprise SSD arrays are billed at* `$450.00/unit`*, whereas* `PO-9004` *specifies* `$400.00/unit`*. The deterministic 3-way matcher immediately flags* `RATE_VARIANCE` *with a* `+$500.00` *(*`INR 43,250.00`*) overcharge and blocks auto-approval in* `REJECTED_DISCREPANCY`*. We can also edit any cell in the line-item table live to see variances recompute dynamically."*
- **Scenario 3 — Critical-Risk Anomaly (`INV-1005`, `Shadow Island Holding BZ`, `$49,500.00` / `INR 4,281,750.00`):**
  - *"Finally, Scenario 3 demonstrates why we pair rules with anomaly scoring.* `INV-1005` *bills* `$49,500` *and matches* `PO-9005` *on price and quantity. However, the 7-D Isolation Forest scores it at* `99.0/100 (CRITICAL)` *because of the combination of an*

offshore jurisdiction (`BZ`*), a* `Net-1` *rush payment window, and an amount just under* `$50,000`*. The perturbation bars show those exact features driving the score, and the router places the invoice in* `FROZEN_FRAUD_RISK`*."*

4. Testing and Engineering Challenges (2 Minutes — Slides 9–10)

- *"In the terminal, running* `python -m pytest tests -q` *verifies that all collected tests pass (*`28 passed` *in Day 29 and* `33 passed` *in Day 30), covering matching math, regex masking, error handling, and dynamic multi-line API requests."*
- *"Two key engineering challenges we solved were: (1) decoupling the frontend line-item table from static preset objects via* `collectLineItemsFromDOM()` *so every user edit is evaluated dynamically, and (2) applying* ***ln(1 + Total$_{USD}$)*** *and* `StandardScaler` *so large dollar amounts do not overshadow bounded variance ratios."*

5. Limitations and Future Work (1–2 Minutes — Slides 11–12)

- *"To conclude, we want to be clear about the current prototype's limitations and how we would evolve it for production:"*

1. *"First,* `AUDIT_LEDGER` *is currently an in-memory Python list that resets on restart; production requires durable* ***PostgreSQL*** *storage and* ***Redis*** *idempotency keys."*
2. *"Second, our* `IsolationForest` *is fit on a seeded synthetic baseline of 1,000 records, and its* `0–100` *score is an uncalibrated anomaly indicator. Production deployment requires evaluation and threshold calibration on authorized historical company invoices plus drift monitoring."*
3. *"Third,* `/api/v1/audit/parse-raw` *performs regex invoice-text parsing; integrating optical OCR (*`LayoutLMv3`*) and* ***OAuth2/RBAC*** *security controls are key next steps."*